Guía para explicar y defender Cantera

La mejor forma de defenderlo es seguir una operación de principio a fin y poder mostrar en qué archivo ocurre cada paso. Esta guía usa el código real del proyecto.

1. Presentación inicial, en tus palabras

«Cantera es una aplicación web para administrar un torneo infantil. Permite registrar equipos, jugadores, jornadas, goles e incidencias y generar reportes para el árbitro y de goleadores. Usamos PHP con Symfony, MySQL y Doctrine en el servidor. Las pantallas se generan con Twig y usan HTML, CSS y JavaScript. Los visitantes consultan; los administradores pueden modificar información».

No es una aplicación React ni una API separada. Symfony genera las páginas HTML en el servidor. `frontend/` y `backend/` separan responsabilidades dentro de la misma aplicación, que se inicia con un servidor PHP.

2. Qué hace cada tecnología

Tecnología	Explicación que debes poder dar
PHP	Lenguaje que ejecuta la lógica en el servidor.
Symfony	Framework que organiza rutas, controladores, formularios, seguridad y servicios.
Twig	Plantillas: toma datos del servidor y produce HTML. Escapa los textos para evitar interpretarlos como código.
HTML	Estructura de las pantallas: formularios, tablas, enlaces y botones.
CSS	Colores, tamaños, distribución y adaptación a celular.
JavaScript	Menú, confirmaciones y vista previa de fotos; Turbo y Stimulus aportan interacción.
MySQL	Guarda de forma persistente los datos del torneo y las cuentas.
Doctrine ORM	Relaciona objetos PHP con tablas y ejecuta consultas/escrituras SQL.
Composer	Instala las librerías PHP y genera el autoload que encuentra las clases.
AssetMapper e Importmap	Organizan y publican los archivos CSS y módulos JavaScript.
PHPUnit	Ejecuta pruebas para comprobar que las reglas y flujos funcionan.
Cloudflare Tunnel	Da acceso temporal desde Internet a la aplicación local. No reemplaza un hosting permanente.

Docker no se utiliza y se retiró su configuración. Node y Playwright se usaron como herramientas auxiliares de revisión; no son necesarios para servir esta web.

3. El ejemplo principal: guardar un jugador

Abre `backend/src/Controller/JugadorController.php`, busca el método `new()` y explica el flujo con la pantalla de inscripción abierta al lado.

1. **GET** `/jugador/new`**:** solicita el formulario vacío. La ruta indica a Symfony qué método ejecutar; `IsGranted('ROLE_ADMIN')` exige una cuenta administradora.

2. `new Jugador()`**:** crea un objeto PHP todavía sin guardar en MySQL.

3. `createForm(JugadorType::class, $jugador)`**:** une los campos del formulario con ese objeto. `JugadorType.php` define nombres, fecha, equipo y fotografía.

4. `handleRequest($request)`**:** toma los datos que llegan del navegador. El formulario también verifica el token CSRF y transforma los tipos de datos.

5. `isSubmitted()` **e** `isValid()`**:** comprueban que se envió y que cumple las reglas de la entidad, del formulario y de las extensiones de validación.

6. **Fotografía:** si existe, se valida y `FotografiaUploader` guarda el archivo con un nombre aleatorio. En MySQL se guarda la referencia, no la imagen completa.

7. `persist($jugador)`**:** indica a Doctrine que el objeto nuevo debe guardarse. `flush()`**:** ejecuta los cambios pendientes en la base de datos.

8. **Redirección 303:** devuelve al listado. Esto evita repetir el POST al recargar la página inmediatamente después de guardar.

Si falla la validación, el flujo no llega a `flush()` y vuelve a mostrar el formulario con errores. Las restricciones del navegador ayudan al usuario, pero el servidor las verifica porque una solicitud puede enviarse sin usar el formulario visual.

4. Modelo, vista y controlador

```
Navegador → public/index.php → Symfony identifica la ruta
         → backend/src/Controller
         → entidades/servicios + Doctrine → MySQL
         → frontend/templates (Twig) → HTML → navegador
```

- **Modelo:** entidades, relaciones y lógica de datos. `Jugador.php` describe un jugador; `Equipo.php`, un equipo. Doctrine los relaciona con MySQL.

- **Vista:** los archivos Twig y los recursos CSS/JS de `frontend/`.

- **Controlador:** recibe la solicitud y coordina el formulario, consultas y respuesta.

- **Servicio:** concentra tareas reutilizables. Por ejemplo, los cálculos de reportes están en `EstadisticasTorneo.php`, no copiados en cada pantalla.

No digas que el controlador es la base de datos ni que Twig guarda los registros: cada parte tiene una función distinta.

5. Relaciones y reglas del torneo

Un equipo tiene muchos jugadores. Cada jugador pertenece a un equipo. Un registro de goles enlaza un jugador y una jornada y contiene la cantidad total. Una incidencia pertenece a un jugador y contiene descripción, tarjeta y fechas.

La clave primaria `id` identifica cada fila. Una clave foránea como `equipo_id` apunta al equipo del jugador e impide referenciar uno inexistente. El índice único de `gol` impide duplicar la combinación jugador/jornada.

La lista de goleadores usa `SUM(g.cantidad)`, agrupa por jugador y ordena el total de mayor a menor. No es un número escrito manualmente en la pantalla. La nómina consulta las tarjetas rojas cuya fecha de suspensión coincide exactamente con la jornada seleccionada. No hay acumulación automática de tarjetas amarillas.

6. Seguridad que puedes demostrar

Control	Cómo explicarlo o mostrarlo
Autenticación	Comprueba quién inicia sesión mediante usuario y contraseña.
Autorización	Decide qué puede hacer: ROLE_ADMIN para crear, editar y eliminar.
Hash	La contraseña no se guarda legible. El login verifica el hash; no se descifra.
CSRF	Token que ayuda a rechazar solicitudes de modificación no legítimas. No es una contraseña ni sustituye al login.
Límite de intentos	Después de fallos repetidos el acceso se bloquea temporalmente. No lo demuestres antes de necesitar entrar en clase.
Validación	Rechaza nombres numéricos, menores de 6 años, fechas inexistentes, equipos inválidos y fotos no permitidas.
SQL parametrizado	Se envían los valores por parámetros de Doctrine, separados de la consulta.
Escape HTML	Twig muestra los textos como texto, sin ejecutar etiquetas inyectadas.
Fotos privadas en disco	Se guardan fuera de `public/` y se entregan mediante una ruta controlada. Su consulta web es pública según el diseño actual.
HTTPS	El enlace Cloudflare cifra la conexión externa. El servidor local usa HTTP en loopback.

La clave sencilla que solicitaste para admin sigue siendo un riesgo real. No la presentes como una medida segura. El sistema no tiene MFA ni protección absoluta contra todos los ataques; los límites están documentados en la auditoría.

7. Demostración de 8–10 minutos

1. Abre el inicio como visitante y explica que las cifras provienen de MySQL.

2. Intenta abrir `/jugador/new` sin sesión: debe enviarte al login.

3. Inicia sesión. Muestra que aparecen las acciones de administración.

4. En inscripción, prueba `12345` como nombre y una fecha con menos de 6 años. Explica el rechazo. Corrige con un nombre ficticio válido y una fecha de 2016.

5. Guarda un jugador de demostración en un equipo existente y abre su ficha. No uses datos personales reales para ensayar.

6. Muestra un total de goles por jugador/jornada. Explica por qué se edita el registro existente en vez de duplicarlo.

7. Muestra una roja sin fecha de suspensión y el error. Usa los datos Demo para comparar una jornada con suspensión y otra sin ella.

8. Abre el reporte de goleadores y el de árbitro; muestra Imprimir/guardar PDF.

9. Abre el código del flujo `new()` y señala entidad, formulario, controlador y Twig.

10. Cierra sesión y comprueba que ya no puedes editar.

Practica antes. Si creas registros de ensayo, usa nombres identificables y elimínalos solo si no tienen dependencias. No borres registros del torneo para resolver un error durante la exposición.

8. Preguntas que podrían hacerte

¿Por qué dos archivos .env? `.env` contiene valores base sin secretos; `.env.local` contiene la configuración privada de esta computadora y tiene prioridad en el uso normal. Las pruebas se configuran en `phpunit.dist.xml` y su conexión privada opcional está dentro de `tests/.env.local`.

¿Por qué no guardar la contraseña en JavaScript? Porque todo lo que se envía al navegador puede inspeccionarse. La autenticación y la comprobación del hash deben hacerse en el servidor.

¿Por qué no se puede borrar un equipo con jugadores? Para conservar la integridad de sus relaciones. Primero se reasignan o resuelven los jugadores dependientes. No se borran automáticamente sus historiales.

¿Cuál es la diferencia entre GET y POST? GET consulta o muestra formularios; POST envía operaciones de cambio en esta aplicación. Usar POST no garantiza seguridad por sí solo: también se comprueban permisos, CSRF y datos.

¿Qué es una migración? Un archivo versionado que modifica el esquema de la base de forma controlada. No es un respaldo de datos.

¿Qué pasa al apagar la computadora? El acceso por el túnel deja de funcionar. Los datos siguen guardados en MySQL. Al iniciar de nuevo, se arrancan MySQL, PHP y el túnel; el enlace temporal puede cambiar.

¿Qué significa que pasen las pruebas? Que se verificaron los casos escritos, no que cualquier situación posible esté cubierta. La suite tiene casos de acceso, validación, archivos, reportes e integración con MySQL.

¿Dónde están las fotos? En `var/uploads/jugadores`. MySQL conserva el nombre del archivo, no la imagen. Por eso un respaldo completo necesita SQL y fotos.

¿Qué mejorarías? Contraseña fuerte, actualización de PHP, alojamiento permanente, paginación para grandes volúmenes y copias automáticas con restauración comprobada.

9. Archivos que debes localizar sin ayuda

- `backend/src/Controller/JugadorController.php`: flujo de inscripción y edición.

- `backend/src/Entity/Jugador.php`: restricciones y relaciones del jugador.

- `backend/src/Form/JugadorType.php`: definición de los campos y la fotografía.

- `frontend/templates/jugador/_form.html.twig`: presentación del formulario.

- `backend/src/Service/EstadisticasTorneo.php`: cálculos de reportes.

- `config/packages/security.yaml`: login, permisos y límite de intentos.

- `frontend/assets/styles/`: diseño visual.

Ensayo recomendado: explica en voz alta cómo llega un nombre desde el formulario hasta MySQL y cómo vuelve a aparecer en el listado. Si puedes seguir ese recorrido y responder por qué se valida en el servidor, ya estás explicando el sistema con comprensión y no solo memorizando nombres de tecnologías.